Retrieval-Augmented Generation (RAG)

Instead of depending only on static training data, RAG grounds outputs in current, state information by combining LLM generation with knowledge base retrieval.
By adding real-time or domain-specific context to prompts, this hybrid approach lessens hallucinations, or fake responses.
It excels in applications that require new or specialized knowledge, such as chatbots, search engines, and enterprise tools.

RAG Process

The documents we want the system to use as knowledge are first gathered. That could be internal company documents, text files, or PDFs.

The documents are then separated into more manageable sections. Because smaller sections increase search accuracy and large language models cannot process very large text at once, this is done.

Each section is split and then transformed into embeddings. Numerical representations of text that convey its meaning are called embeddings. After that, they are kept in a vector database.

When a user submits a query, the system turns it into an embedding and looks up related content in the database. The most pertinent passages are obtained and provided as context for the language model. Lastly, the model uses the retrieval to produce an answer.

RAG Types

1. **Basic RAG**
   This is the most simple variant, in which documents are delivered straight to the model after being retrieved once.
2. **Advanced RAG**
   Additi4onal processing is carried out in this category, such as filtering, reranking obtained documents, or merging different retrieval techniques.
3. **RAG hybrid**
   To increase accuracy, this blends semantic and keyword-based search.

How RAG Works

Retrieval and generation are the two primary processes that RAG combines. The system first looks for related information before responding directly from memory. It then makes use of that data to generate a response that is more precise and significant. As a result, the response is more trustworthy and supported by real data.

Important Concepts in RAG

Chunking
Splits large documents into smaller pieces to improve retrieval accuracy.

Embedding
Converts text into numerical vectors that capture meaning. Similar meaning → similar vectors.

Vector Database
Specialized database for storing and searching embeddings efficiently.

Similarity Search
Finds semantically similar content using vector distance (e.g., cosine similarity).

Prompt Augmentation
Adds retrieved context to the user query before sending it to the LLM.

Retriever
Component responsible for selecting the most relevant chunks.

My Understanding of RAG

From my understanding, RAG makes AI systems more practical and reliable by connecting them to real data sources. Instead of generating responses purely from trained knowledge, the model retrieves relevant information and then generates context-aware answers. This approach significantly improves accuracy and makes AI suitable for enterprise-level and domain-specific applications.